

LAPORAN PRAKTIKUM
ALGORITMA DAN PEMROGRAMAN 2
MODUL 17
“SKEMA PEMROSESAN SEKUENSIAL”



DISUSUN OLEH:
Dharma Chandra Viriya
109082500052
S1 IF-13-02
DOSEN:
Wahyu Andi Saputra, S.Pd., M.Eng.

PROGRAM STUDI S1 TEKNIK INFORMATIKA
FAKULTAS INFORMATIKA
TELKOM UNIVERSITY PURWOKERTO
2024/2025

DASAR TEORI

CONTOH SOAL

1. Latihan1

Source Code:

Output:

Deskripsi Program:

SOAL LATIHAN

Statement skema pemrosesan sekuensial

1. Latihan1

Source Code:

```
package main

import "fmt"

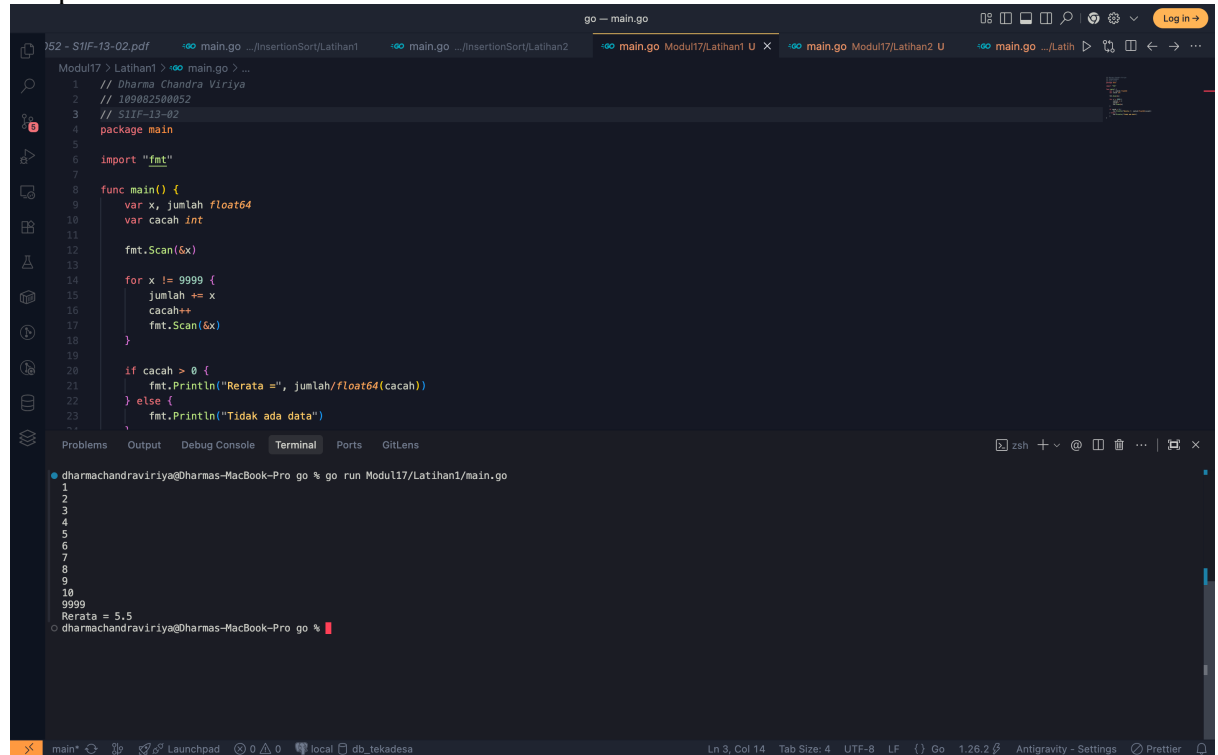
func main() {
    var x, jumlah float64
    var cacah int

    fmt.Scan(&x)

    for x != 9999 {
        jumlah += x
        cacah++
        fmt.Scan(&x)
    }

    if cacah > 0 {
        fmt.Println("Rerata =", jumlah/float64(cacah))
    } else {
        fmt.Println("Tidak ada data")
    }
}
```

Output:



```
1 // Dharma Chandra Viriya
2 // 109082500052
3 // S11F-13-02
4 package main
5
6 import "fmt"
7
8 func main() {
9     var x, jumlah float64
10    var cacah int
11
12    fmt.Scan(&x)
13
14    for x != 9999 {
15        jumlah += x
16        cacah++
17        fmt.Scan(&x)
18    }
19
20    if cacah > 0 {
21        fmt.Println("Rerata =", jumlah/float64(cacah))
22    } else {
23        fmt.Println("Tidak ada data")
24    }
25 }
```

dharmachandraviriya@Dharmas-MacBook-Pro % go run Modul17/Latihan1/main.go

```
1
2
3
4
5
6
7
8
9
10
9999
Rerata = 5,5
dharmachandraviriya@Dharmas-MacBook-Pro %
```

Deskripsi Program:

Program ini dibuat untuk menghitung rerata dari sejumlah bilangan real yang dimasukkan oleh pengguna. Data dibaca secara sekuensial hingga ditemukan nilai 9999 sebagai marker atau penanda akhir input. Nilai marker tidak ikut dihitung dalam proses perhitungan rerata. Jika tidak ada data yang dimasukkan sebelum marker, program akan menampilkan pesan "Tidak ada data". Secara teknis, program menggunakan variabel jumlah untuk menyimpan total seluruh bilangan yang valid dan variabel cacah untuk menghitung banyaknya data yang dibaca. Data pertama dibaca sebelum perulangan, kemudian selama nilai yang dibaca bukan 9999, program akan menambahkan nilai tersebut ke jumlah, menambah nilai cacah, dan membaca data berikutnya. Setelah perulangan selesai, rerata dihitung dengan rumus jumlah / float64(cacah) dan ditampilkan ke layar. Pola yang digunakan merupakan skema pemrosesan sekuensial dengan marker, di mana pembacaan data dilakukan terus-menerus hingga ditemukan penanda akhir data.

2. Latihan2

Source Code:

```
package main

import "fmt"

func main() {
    var x, data string
    var n, i int
    var jumlah int = 0
    var posisi int = -1

    fmt.Scan(&x)
    fmt.Scan(&n)

    for i = 1; i <= n; i++ {
        fmt.Scan(&data)

        if data == x {
            jumlah++

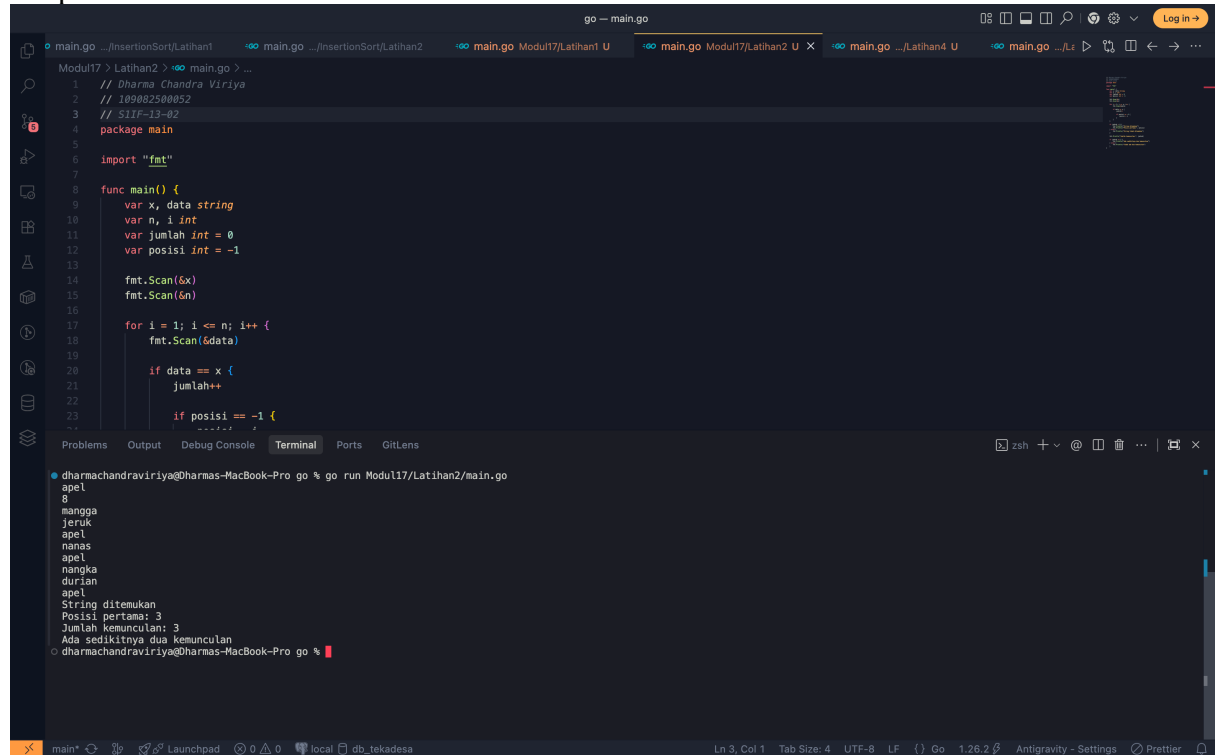
            if posisi == -1 {
                posisi = i
            }
        }
    }

    if jumlah > 0 {
        fmt.Println("String ditemukan")
        fmt.Println("Posisi pertama:", posisi)
    } else {
        fmt.Println("String tidak ditemukan")
    }

    fmt.Println("Jumlah kemunculan:", jumlah)

    if jumlah >= 2 {
        fmt.Println("Ada sedikitnya dua kemunculan")
    } else {
        fmt.Println("Tidak ada dua kemunculan")
    }
}
```

Output:



```
go -- main.go
Modul17 > Latihan2 > main.go > ...
1 // Dharma Chandra Viriya
2 // 109082500052
3 // SIIIF-13-02
4 package main
5
6 import "fmt"
7
8 func main() {
9     var x, data string
10    var n, i int
11    var jumlah int = 0
12    var posisi int = -1
13
14    fmt.Scan(&x)
15    fmt.Scan(&n)
16
17    for i = 1; i <= n; i++ {
18        fmt.Scan(&data)
19
20        if data == x {
21            jumlah++
22
23            if posisi == -1 {
24                posisi = i
25            }
26        }
27    }
28
29    fmt.Println("String ditemukan")
30    fmt.Println("Posisi pertama: ", posisi)
31    fmt.Println("Jumlah kemunculan: ", jumlah)
32    fmt.Println("Ada sedikitnya dua kemunculan")
33 }
```

dharmachandraviriya@Dharmas-MacBook-Pro % go run Modul17/Latihan2/main.go

```
apel
8
mangga
jeruk
apel
mangga
apel
mangga
durian
apel
String ditemukan
Posisi pertama: 3
Jumlah kemunculan: 3
Ada sedikitnya dua kemunculan
dharmachandraviriya@Dharmas-MacBook-Pro %
```

Deskripsi Program:

Program ini dibuat untuk melakukan pencarian sebuah string pada sekumpulan data string yang dimasukkan oleh pengguna. Program menerima sebuah string x sebagai data yang akan dicari, kemudian membaca n buah string sebagai kumpulan data. Setelah proses pencarian selesai, program akan menampilkan apakah string x ditemukan atau tidak, posisi pertama kemunculannya, jumlah kemunculannya dalam kumpulan data, serta informasi apakah string tersebut muncul sedikitnya dua kali. Secara teknis, program menggunakan perulangan sebanyak n kali untuk membaca setiap string dan membandingkannya dengan string x. Variabel jumlah digunakan untuk menghitung banyaknya kemunculan string yang dicari, sedangkan variabel posisi digunakan untuk menyimpan posisi pertama kemunculan string tersebut. Nilai awal posisi diisi -1 sebagai penanda bahwa string belum ditemukan. Setiap kali ditemukan string yang sama dengan x, nilai jumlah akan bertambah satu. Jika itu merupakan kemunculan pertama, posisi indeks saat itu akan disimpan ke dalam variabel posisi. Setelah seluruh data diproses, program menampilkan hasil pencarian berdasarkan nilai jumlah dan posisi. Pendekatan ini memungkinkan seluruh informasi yang diminta diperoleh hanya dengan satu kali penelusuran data sehingga proses pencarian menjadi lebih efisien.

3. Latihan3

Source Code:

```
package main

import (
    "fmt"
    "math/rand"
)

func main() {
    var n int
    var A, B, C, D int
    var x, y float64

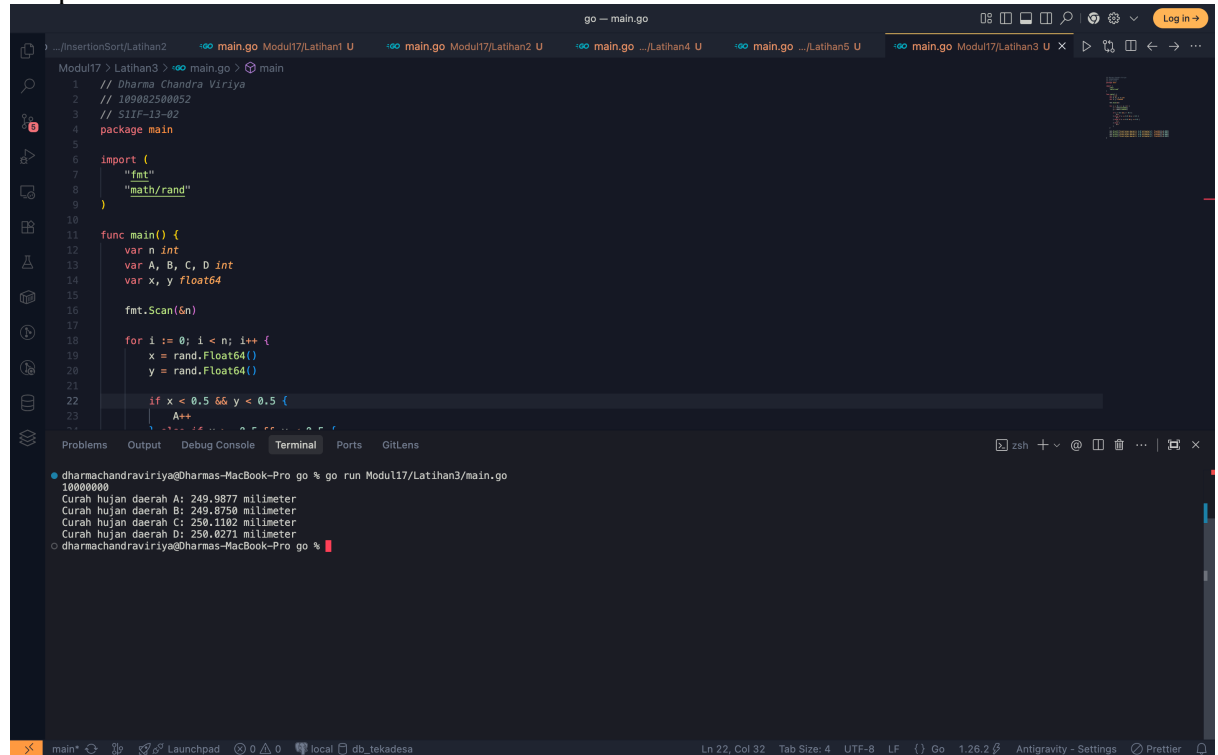
    fmt.Scan(&n)

    for i := 0; i < n; i++ {
        x = rand.Float64()
        y = rand.Float64()

        if x < 0.5 && y < 0.5 {
            A++
        } else if x >= 0.5 && y < 0.5 {
            B++
        } else if x >= 0.5 && y >= 0.5 {
            C++
        } else {
            D++
        }
    }

    fmt.Printf("Curah hujan daerah A: %.4f milimeter\n", float64(A)*0.0001)
    fmt.Printf("Curah hujan daerah B: %.4f milimeter\n", float64(B)*0.0001)
    fmt.Printf("Curah hujan daerah C: %.4f milimeter\n", float64(C)*0.0001)
    fmt.Printf("Curah hujan daerah D: %.4f milimeter\n", float64(D)*0.0001)
}
```


Output:



```
1 // Dharma Chandra Viriya
2 // 109002500052
3 // SIIIF-13-02
4 package main
5
6 import (
7     "fmt"
8     "math/rand"
9 )
10
11 func main() {
12     var n int
13     var A, B, C, D int
14     var x, y float64
15
16     fmt.Scan(&n)
17
18     for i := 0; i < n; i++ {
19         x = rand.Float64()
20         y = rand.Float64()
21
22         if x < 0.5 && y < 0.5 {
23             A++
24         }
25     }
26
27     fmt.Printf("Curah hujan daerah A: %d, 0001 milimeter\n", A)
28     fmt.Printf("Curah hujan daerah B: %d, 0001 milimeter\n", B)
29     fmt.Printf("Curah hujan daerah C: %d, 0001 milimeter\n", C)
30     fmt.Printf("Curah hujan daerah D: %d, 0001 milimeter\n", D)
31 }
```

```
dharmachandraviriya@Dharmas-MacBook-Pro % go run Modul17/Latihan3/main.go
10000000
Curah hujan daerah A: 249,9877 milimeter
Curah hujan daerah B: 249,8750 milimeter
Curah hujan daerah C: 250,1102 milimeter
Curah hujan daerah D: 250,0271 milimeter
dharmachandraviriya@Dharmas-MacBook-Pro %
```

Deskripsi Program:

Program ini dibuat untuk mensimulasikan pengukuran curah hujan pada empat daerah, yaitu A, B, C, dan D. Setiap tetesan hujan direpresentasikan sebagai sebuah titik acak (x, y) yang berada pada bidang dengan koordinat dari 0 hingga 1. Pengguna memasukkan banyaknya tetesan hujan yang akan disimulasikan, kemudian program menghitung jumlah tetesan yang jatuh pada masing-masing daerah. Setiap tetesan dianggap memiliki curah hujan sebesar 0,0001 milimeter, sehingga hasil akhir ditampilkan dalam satuan milimeter untuk setiap daerah. Secara teknis, program menggunakan fungsi `rand.Float64()` dari paket `math/rand` untuk menghasilkan koordinat acak x dan y pada interval [0,1). Bidang dibagi menjadi empat bagian yang sama besar menggunakan garis $x = 0,5$ dan $y = 0,5$. Program melakukan perulangan sebanyak n kali untuk mensimulasikan setiap tetesan hujan. Pada setiap iterasi, koordinat titik diperiksa menggunakan struktur percabangan untuk menentukan apakah tetesan tersebut berada pada daerah A, B, C, atau D. Jumlah tetesan pada masing-masing daerah disimpan dalam variabel A, B, C, dan D. Setelah seluruh simulasi selesai, banyaknya tetesan pada setiap daerah dikonversi menjadi curah hujan dengan mengalikan jumlah tetesan dengan 0,0001, kemudian hasilnya ditampilkan menggunakan `fmt.Printf` dengan empat angka di belakang koma. Program ini menerapkan konsep simulasi berbasis bilangan acak (random simulation) untuk memperkirakan distribusi curah hujan pada beberapa wilayah.

4. Latihan4

Source Code:

```
package main

import (
    "fmt"
    "math"
)

func main() {
    var jumlah float64
    var i int

    for i = 1; ; i++ {

        si := 1.0 / float64(2*i-1)
        if i%2 == 0 {
            si = -si
        }

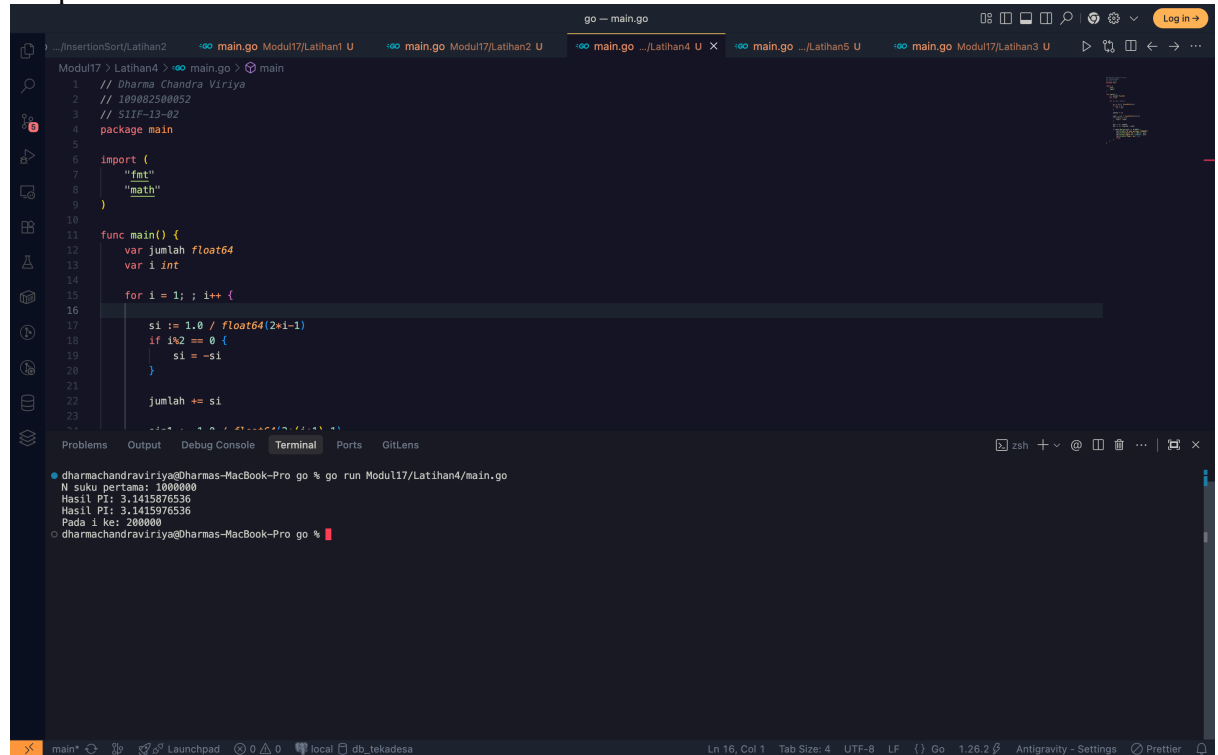
        jumlah += si

        sip1 := 1.0 / float64(2*(i+1)-1)
        if (i+1)%2 == 0 {
            sip1 = -sip1
        }

        pi1 := 4 * jumlah
        pi2 := 4 * (jumlah + sip1)

        if math.Abs(pi1-pi2) <= 0.00001 {
            fmt.Println("N suku pertama: 1000000")
            fmt.Printf("Hasil PI: %.10f\n", pi1)
            fmt.Printf("Hasil PI: %.10f\n", pi2)
            fmt.Println("Pada i ke:", i)
            break
        }
    }
}
```

Output:



```
1 // Dharma Chandra Viriya
2 // 109002500052
3 // S11F-13-02
4 package main
5
6 import (
7     "fmt"
8     "math"
9 )
10
11 func main() {
12     var jumlah float64
13     var i int
14     for i = 1; i++ {
15         si := 1.0 / float64(2*i-1)
16         if i%2 == 0 {
17             si = -si
18         }
19         jumlah += si
20     }
21     fmt.Println("N suku pertama: 1000000")
22     fmt.Println("Hasil PI: 3.1415976536")
23     fmt.Println("Hasil PI: 3.1415976536")
24     fmt.Println("Pada i ke: 200000")
25 }
```

dharmachandraviriya@Dharmas-MacBook-Pro % go run Modul17/Latihan4/main.go

N suku pertama: 1000000
Hasil PI: 3.1415976536
Hasil PI: 3.1415976536
Pada i ke: 200000

Deskripsi Program:

Program ini dibuat untuk menghitung nilai konstanta π menggunakan deret Leibniz, yaitu deret harmonik ganti yang secara matematis konvergen ke nilai $\pi/4$. Program melakukan penjumlahan suku-suku deret secara bertahap dan terus berlanjut hingga selisih antara dua pendekatan nilai π yang berurutan tidak lebih dari 0,00001. Setelah kondisi tersebut terpenuhi, program menampilkan dua nilai pendekatan π yang bersebelahan serta nilai indeks i saat proses dihentikan. Secara teknis, program menggunakan variabel `jumlah` untuk menyimpan akumulasi hasil penjumlahan deret dan variabel `i` sebagai pencacah suku. Pada setiap iterasi, program menghitung suku ke- i (si) dengan penyebut bilangan ganjil dan tanda yang berganti antara positif dan negatif. Nilai si kemudian ditambahkan ke `jumlah`. Selanjutnya program menghitung suku berikutnya ($si1$) untuk memperoleh dua pendekatan nilai π , yaitu $pi1 = 4 \times \text{jumlah}$ dan $pi2 = 4 \times (\text{jumlah} + si1)$. Fungsi `math.Abs()` digunakan untuk menghitung nilai mutlak selisih antara $pi1$ dan $pi2$. Apabila selisih tersebut kurang dari atau sama dengan 0,00001, perulangan dihentikan dan kedua nilai pendekatan π beserta nilai i ditampilkan ke layar. Program ini menerapkan konsep perulangan tak hingga (infinite loop), deret tak hingga, dan kriteria konvergensi untuk memperoleh pendekatan nilai π dengan tingkat ketelitian tertentu.

5. Latihan5

Source Code:

```
package main

import (
    "fmt"
    "math/rand"
)

func main() {
    var n int
    var dalam int
    var pi float64

    fmt.Print("Banyak Topping: ")
    fmt.Scan(&n)

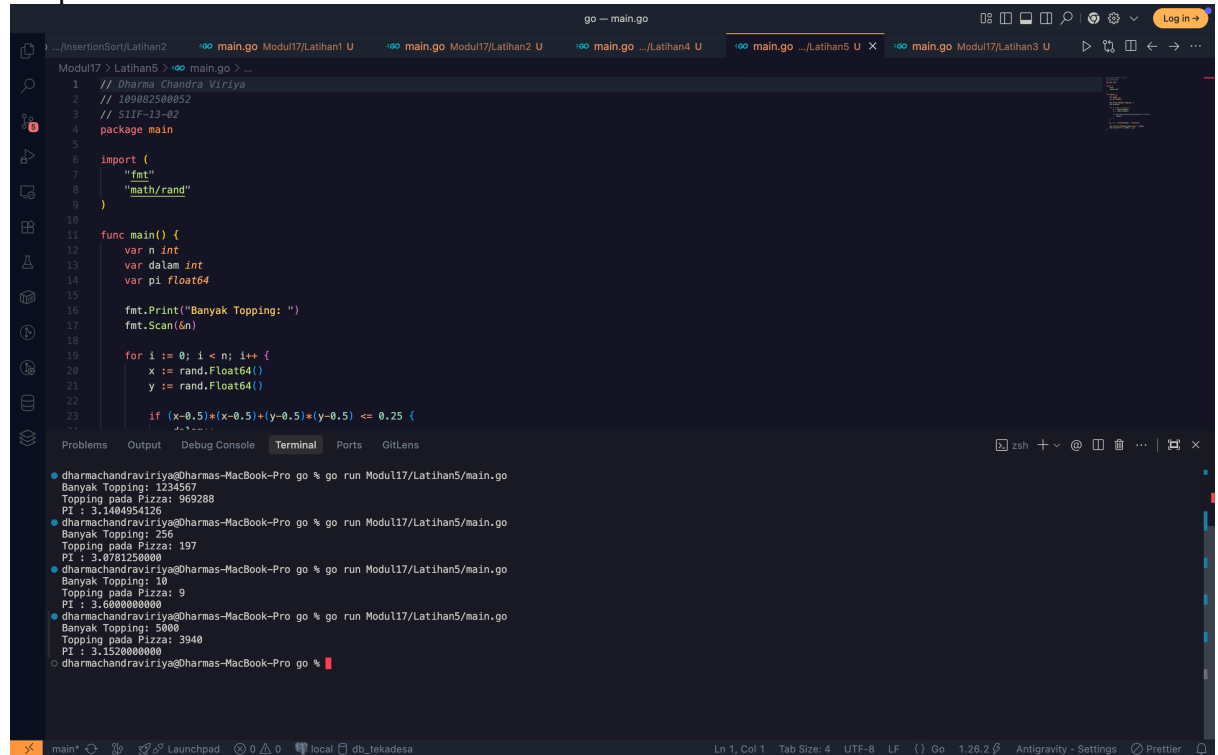
    for i := 0; i < n; i++ {
        x := rand.Float64()
        y := rand.Float64()

        if (x-0.5)*(x-0.5)+(y-0.5)*(y-0.5) <= 0.25 {
            dalam++
        }
    }

    pi = 4 * float64(dalam) / float64(n)

    fmt.Println("Topping pada Pizza:", dalam)
    fmt.Printf("PI : %.10f\n", pi)
}
```

Output:



```
go -- main.go
Modul17 > Latihan5 > go run Modul17/Latihan5/main.go
1 // Dharma Chandra Viriya
2 // 109002500052
3 // S11F-13-02
4 package main
5
6 import (
7     "fmt"
8     "math/rand"
9 )
10
11 func main() {
12     var n int
13     var dalam int
14     var pi float64
15
16     fmt.Println("Banyak Topping: ")
17     fmt.Scan(&n)
18
19     for i := 0; i < n; i++ {
20         x := rand.Float64()
21         y := rand.Float64()
22         if (x-0.5)*(x-0.5)+(y-0.5)*(y-0.5) <= 0.25 {
23             dalam++
24         }
25     }
26     pi = 4 * float64(dalam) / float64(n)
27     fmt.Printf("Topping pada Pizza: %d\n", n)
28     fmt.Printf("PI : %.10f\n", pi)
29 }
dharma@Dharma-MacBook-Pro: ~/go/src/modul17/latihan5 $ go run Modul17/Latihan5/main.go
Banyak Topping: 1234567
Topping pada Pizza: 969288
PI : 3.1404954126
dharma@Dharma-MacBook-Pro: ~/go/src/modul17/latihan5 $ go run Modul17/Latihan5/main.go
Banyak Topping: 256
Topping pada Pizza: 197
PI : 3.0781250000
dharma@Dharma-MacBook-Pro: ~/go/src/modul17/latihan5 $ go run Modul17/Latihan5/main.go
Banyak Topping: 10
Topping pada Pizza: 9
PI : 3.6000000000
dharma@Dharma-MacBook-Pro: ~/go/src/modul17/latihan5 $ go run Modul17/Latihan5/main.go
Banyak Topping: 5000
Topping pada Pizza: 3940
PI : 3.1520000000
dharma@Dharma-MacBook-Pro: ~/go/src/modul17/latihan5 $
```

Deskripsi Program:

Program ini dibuat untuk mensimulasikan proses penaburan topping pada sebuah pizza berbentuk lingkaran yang berada di dalam wadah berbentuk persegi, sekaligus menghitung pendekatan nilai konstanta π menggunakan metode Monte Carlo. Pengguna memasukkan banyaknya topping yang akan ditaburkan, kemudian program menghasilkan titik-titik acak sebagai posisi topping. Setelah seluruh titik diproses, program menampilkan jumlah topping yang jatuh tepat di dalam area pizza serta nilai pendekatan π yang diperoleh dari hasil simulasi tersebut. Secara teknis, program menggunakan fungsi `rand.Float64()` dari paket `math/rand` untuk menghasilkan koordinat acak (x, y) pada interval $[0,1)$. Pizza dimodelkan sebagai lingkaran dengan titik pusat $(0,5; 0,5)$ dan jari-jari $0,5$, sehingga sebuah titik dianggap berada di dalam pizza jika memenuhi persamaan $(x - 0,5)^2 + (y - 0,5)^2 \leq 0,25$. Program melakukan perulangan sebanyak n kali untuk menghasilkan setiap titik acak dan menggunakan variabel `dalam` untuk menghitung jumlah titik yang berada di dalam lingkaran. Setelah seluruh simulasi selesai, nilai π dihitung menggunakan rumus $\pi = 4 \times (\text{jumlah titik dalam lingkaran} / \text{jumlah seluruh titik})$, yang berasal dari perbandingan luas lingkaran terhadap luas persegi pembungkusnya. Hasil akhir berupa jumlah topping pada pizza dan nilai pendekatan π ditampilkan menggunakan fungsi `fmt.Println` dan `fmt.Printf` dengan presisi sepuluh angka di belakang koma. Program ini menerapkan konsep simulasi acak (Monte Carlo Simulation) dan geometri bidang untuk mengestimasi nilai konstanta π .

DAFTAR PUSTAKA